

第1章

はじめての Cloudflare Workers

1.1 Cloudflare Workersとは

Cloudflare Workersの概要

Cloudflare Workersは、Cloudflareが展開しているサーバーレスコンピューティングのプラットフォームです。Workerは、JavaScriptの実行エンジンであるV8 (Google Chrome/Node.jsで使われているもの) を使用して、JavaScriptのコードおよび、それを通じて読み込まれたWebAssemblyのバイナリを動かします。Cloudflare WorkersはいわゆるFunctions as a Service (FaaS) の1つとして見ることができ、類似のサービスとしてはAWS Lambda、Cloud Functions (Google Cloud)、Azure Functionsなどが挙げられます。しかし、Cloudflare Workersは、それ単体で取り上げて単なるFaaSとして扱うより、Cloudflareプラットフォームの製品の中核としてとらえるほうがより適切です。

1.2では、Cloudflare Workersの特徴とユースケース、およびWorkerと連携できるCloudflareプラットフォームの製品についての紹介を行います。

Cloudflare Workersの特徴

それでは、さっそくCloudflare Workersがいったいどのような特徴を備えているのかを見てみましょう。

0ミリ秒コールドスタート

一般的なWebアプリケーションが1台～数台のインスタンスを起動し続けた状態でリクエストを受け付け続けるのに対し、サーバーレスコンピューティングでは基本的にリクエストを受け付けてからアプリケーションの起動を行い、リクエストを処理したうえでレスポンスを返します。ここで言う“リクエストを受け付けてからリクエストを処理するまで”の時間が発生してしまう点は、インスタンスを常時起動するアプリケーションに対して不利な点と言えます。

例として、あるアプリケーションが、起動に1秒、リクエストの処理に0.1秒かかるという単純なケースで考えてみましょう。常時起動のアプリケーションでは、リクエストを受け付けた時点ですでに起動が完了しているので、レスポンスまでに0.1秒だけかかります。しかしながら、リクエストを受け付けてから起動するアプリケーションでは、レスポンスまでに1.1秒かってしまうことになります。

リクエストを受け付ける頻度の高いアプリケーションの場合、毎回レスポンスに長い時間がかかってしまうようでは不便です。そのため、一般的にサーバーレスコンピューティングでは、一定の期間、起動したアプリケーションをすぐに再開できる状態に維持します。これによって、一度リクエストを受け付けた後の二度目のリクエストに対しては起動のオーバーヘッドなしにレスポンスを返すことができるようになります。このような、アプリケーションがすぐに再開できる状態になっていることを「ウォーム」状態と言います。ただし、このウォーム状態を維持するとなると、結局インスタンスを長時間維持していることと変わらなくなってしまうため、しばらくリクエストがないと解除されます。

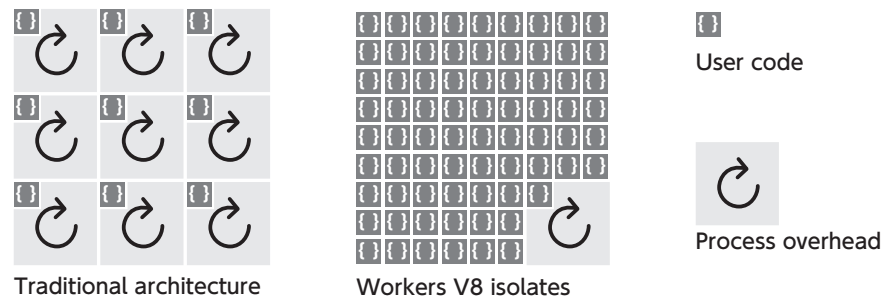
サーバーレスコンピューティングの文脈でよく取り上げられる「コールドスタート」とは、リクエスト受け付け時にウォーム状態のアプリケーションが足りていないときに、アプリケーションの起動から行うケースのことを指します。ユーザーがリクエストを送った際にコールドスタートが発生すると、ウォーム状態のアプリケーションへのリクエストと比較して明確な遅延が発生します。ユーザーがコールドスタートによるレスポンス遅延を感じないようにするテクニックとして、アプリケーションに対し一定の間隔でリクエストを送り続ける手法などが使用されていますが、Cloudflareによると、Cloudflare Workersではこうした工夫を行う必要はないとのこと。なぜなら、Cloudflare Workersはインスタンスが立ち上がっていない状態から、0ミリ秒でコールドスタートを行うことができるからです。

Cloudflare Workersの0ミリ秒コールドスタートは、次の2つの技術の組み合わせによって実現されています。

- V8 Isolate
- TLSハンドシェイク中のウォームアップ

一般的なサーバーレスコンピューティングでは、コンテナ化されたプログラムの実行環境をアプリケーションひとつひとつに対して立ち上げ、その上でアプリケーションを実行します(図1左)。一方で、Cloudflare Workersでは、V8 Isolateと呼ばれる、ユーザーの使用するコードと変数だけを含んだより軽量のコンテキストでアプリケーションの実行を行います。1つのV8のランタイム上で、数百から数千ものV8 Isolateを実行することができるため、従来のモデルに比べてアプリケーションの立ち上げにかかるオーバーヘッドが非常に小さくなります(図1中央)。この機構はV8に強く依存するため、実行できるプログラムがJavaScriptおよびWebAssemblyに限定されるという強い制約はありますが、これによって得られる起動速度は、一般的なサーバーレスコンピューティングのアーキテクチャに対するCloudflare Workers独自の強みとなっています。

図1 従来のアーキテクチャと、Workers V8 isolatesの比較



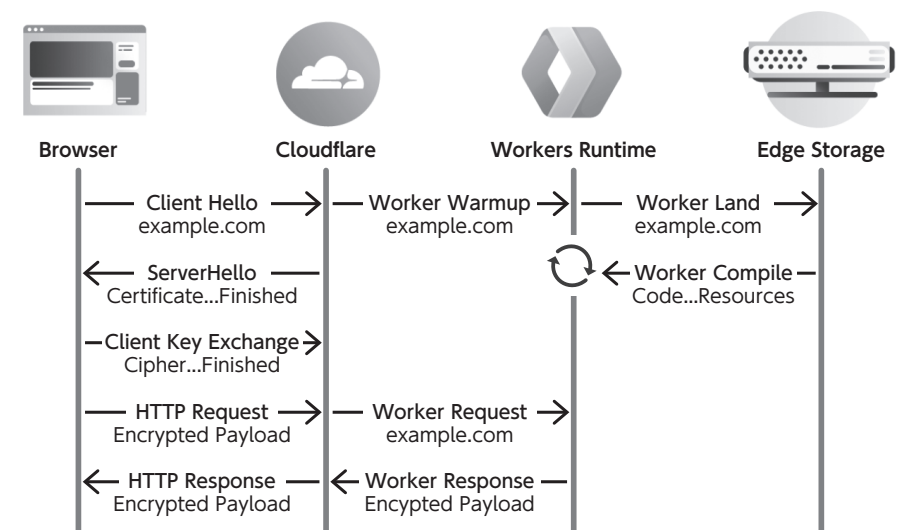
※出典：<https://developers.cloudflare.com/workers/learning/how-workers-works/>

しかし、V8 Isolateによるアプリケーション立ち上げの高速化だけでは、0ミリ秒コールドスタートが実現できたとは言えません。Cloudflareは、さらなる高速化の手段として、TLSハンドシェイク中のウォームアップを行います。この手法は、Workerとクライアント間の通信がHTTPSで行われることを前提としています。現代のWebでは、HTTPSの利用はほぼ必須と言えるので、この前提はほとんどの場合で成り立つでしょう。

Workerを使用するエンドポイントへのリクエストをCloudflareが受け取ると、まずTLSのハンドシェイクを行います(図2左から2番目)。TLSのハンド

シェイクでは、クライアント/サーバ間でのラウンドトリップが発生するため、一定の時間が必要となります。Cloudflare Workersでは、このハンドシェイクが完了する前にWorkerの起動を始めるという最適化を行っています。Workerの起動は5ミリ秒程度で完了するため、TLSハンドシェイクが完了するころにはWorkerがすでに立ち上がっており、0ミリ秒でコールドスタートが完了するというしくみです(図2右から2番目)。

図2 TLSハンドシェイク中のWorkerの起動シーケンス



※出典：<https://blog.cloudflare.com/eliminating-cold-starts-with-cloudflare-workers/>

ただし、0ミリ秒コールドスタートは、そのしくみ上、Workerのコードのサイズによってはうまくいきません。Cloudflareの提供するCLIツールのwranglerで、gzip後のサイズが1MiBを超えるWorkerをデプロイしようとすると、次のような警告が表示されます。

```
> ▲ [WARNING] We recommend keeping your script less than 1MiB (1024 KiB)
after gzip. Exceeding past this can affect cold start time
```

Workerのパフォーマンスを最大限引き出すためには、ある程度サイズを小

さく抑えておいたほうがよいでしょう。

🌀 複数言語のサポート

Cloudflare Workersは、JavaScriptランタイムであるV8を使用しているので、基本的にコードの記述にJavaScriptを使用します。もちろんですが、TypeScriptを使用することもできます。Cloudflareの提供するCLIツールのwranglerはTypeScriptを手厚くサポートしており、自前でトランスパイルの設定を書くことなく、TypeScriptをそのまま書くのに近い体験を提供します。Workerを通じて呼び出せる機能への型定義も充実しています。

また、先に述べたとおりV8はWebAssemblyを実行することができます。Cloudflare Workersはこれを活用してあらゆるプログラミング言語によるWorkerの記述を可能としています。WebAssemblyは、Webブラウザ上であらゆるプログラミング言語からコンパイルしたバイナリを動作させることを目的にWeb標準として策定されましたが、近年では、安全性・可搬性の高いバイナリフォーマットとして、Webブラウザ以外での用途が急速に広まっています。Cloudflare WorkersのようなFaaSでの活用も、Webブラウザ外での用途の代表的な例と言えるでしょう。

Cloudflare Workersのトップページ^{注1}では、特徴としてWorkerをJavaScript/Rust/C/C++によって記述可能であることが挙げられています。ここでのRust/C/C++による記述は、WebAssemblyの利用を前提としています。Rustに関しては、最も手厚くサポートが行われています。RustでWorkerを記述するために作られたworkers-rs crate^{注2}と、これを使ったworker-rustテンプレート^{注3}を使用することで、WebAssemblyをロードするJavaScriptコードの存在を意識することなく開発を行えます。C/COBOLに関しては、WebAssemblyに変換する設定が事前に済んだテンプレートが用意されています。サードパーティー製ではありますが、筆者の作成したsyumai/workers package^{注4}などを使用することで、GoからWebAssemblyに変換したWorker

注1 <https://workers.cloudflare.com/>

注2 <https://github.com/cloudflare/workers-rs>

注3 <https://github.com/cloudflare/workers-sdk/tree/main/templates/experimental/worker-rust>

注4 <https://github.com/syumai/workers>

を動作させることも可能です。

Cloudflareは、WebAssembly以外にも、JavaScriptにトランスパイル可能な言語によるWorkerのテンプレートを豊富にそろえています。現在提供されているのはDart、F#、Kotlin、Perl、PHP、Python、Reason/OCaml、Scalaのテンプレートです^{注5}。

🌀 グローバルな環境へのデプロイ

Workerの実行は、Cloudflareのデータセンター上で行われます。このデータセンターは世界100カ国以上、300都市に展開されており、世界のインターネット人口の95%が50ミリ秒以内で到達可能となっています（日本でデータセンターが設置されているのは、東京、大阪、福岡、那覇の4カ所です）。Workerは、リクエストを行ったユーザーから近い位置で実行されるため、一度デプロイしただけで、すぐに全世界から高速にアクセス可能となります。そのため、Cloudflare Workersを利用するうえでは、基本的にリージョンを意識する必要はありません。ただし、Cloudflare Workersをプロキシ用途で使用する場合は、プロキシ先のオリジンサーバとの距離を考慮する必要があります。

🌀 Cloudflareプラットフォーム製品との柔軟な連携

実用的なWebアプリケーションの構築には、データベースや、静的コンテンツの配信用ストレージなどが必要となります。Cloudflareは、自身のプラットフォームの製品をWorkerを通じて利用可能な形で提供しています。代表的なものをいくつか紹介します。

Cloudflare R2

Cloudflare R2（以下、R2）は、Amazon S3互換のストレージサービスです。S3に存在する機能をすべて実装しているわけではありませんが、APIにある程度互換性があるため、既存のS3向けに開発されたライブラリなどを活用できます。

R2の特徴は、何よりその価格体系にあります。なんと、R2はEgress（外向き）

注5 <https://developers.cloudflare.com/workers/learning/languages/#compile-languages-to-javascript>

のネットワーク転送に対しての料金がかかりません。料金がかかるのは、ストレージの容量（1GBあたり毎月0.015ドル）と、ストレージのオブジェクトへの操作（作成・取得など）だけです。S3からR2に移行することで、大幅なコストカットに成功した事例も徐々に出てきています。TypeScript向けORMのPrismaは、そのエンジンの配信元のストレージをS3からR2へ切り替えたことで、98%のコストカットに成功しています^{注6}。

もちろん、R2の機能はWorkerから簡単に呼び出すことができますが、Workerと関係なく、それ単体としても非常に魅力的なサービスとなっています。ただし、データがどのリージョンに保存されるかの制御を完全に行うことはできないという点には注意が必要です。基本的に、バケットが作成されるリージョンとしては利用可能な最も近い距離のものが自動的に選択されますが、明示的にロケーションヒントを与えることも可能となっています^{注7}。

Cloudflare Workers KV

Cloudflare Workers KV（以下、KV）は、Workerから利用可能な分散キーバリューストアです。KVは、結果整合性のモデルを採用しており、書き込まれたデータが即時に読み取り可能となる保証がありません。その代わり、全世界のロケーションでのキャッシュに重点を置いており、キャッシュが更新された後の読み取りは高速に行われます。KVは書き込みが低頻度で、読み取りが高頻度となる使い方に向いています（本稿では紹介を割愛しますが、強整合性が必要な場合はDurable Objectsという別の製品を使用します）。各ロケーションでのキーに対応するデータのキャッシュ保持期間はデフォルトで60秒となっていますが、KVからのデータ取得時にcacheTTLパラメータを設定することで、より長い時間キャッシュを維持することも可能です。キャッシュの保持期間が長すぎた場合、cacheTTLを小さい数字に設定してデータを取得することで、後からより短いTTLに変更することも可能です。このように、KVではキャッシュの設定を柔軟に行えます。KVは、1つのキーあたりに最大25MiBのデータを格納可能ですので、文字情報だけでなく、画像などのキャッシュにも活用できます。

^{注6} <https://blog.cloudflare.com/how-prisma-saved-98-percent-on-distribution-costs-with-cloudflare-r2/>

^{注7} <https://developers.cloudflare.com/r2/reference/data-location/>

Cache API

Cloudflareのキャッシュを操作するためのCache APIも利用可能です。Cache APIは、Web標準のCache APIに則り、非常に柔軟な形で提供されています。KVと比較して制限が非常に緩く、1つのオブジェクトにつき最大512MBまで保持可能で、1リクエストごとのキャッシュの作成は合計で5GBまで可能です。これだけの大容量のキャッシュが、なんと無料プランから使用できてしまいます。全世界から読み取られることを想定した、動的に変化する値を格納したいケースではKVが適していますが、単に各ロケーションごとにキャッシュを保持しておきたいという用途では、Cache APIの利用が適しているでしょう。

Cloudflare D1

リレーショナルデータベースを使用したい場合は、Cloudflare D1（以下、D1）が利用可能です。D1は、SQLiteをベースに構築されたサーバーレスのデータベースです。D1の特徴は、リードレプリカの自動作成です。各ロケーションに、リードレプリカが自動的に作成され、変更が反映されていくため、世界中で読み取りが高速に行えることが期待できます。ただし、2023年9月に発表されたオープンベータの時点^{注8}では、まだリードレプリカの作成機能は開発中とアナウンスが出ています。まだD1自体はベータ版ではありますが、TypeScript向けのORMであるDrizzle ORMや、Kyselyへのアダプターなど、コミュニティによるプロジェクトが活発に開発されており、今後に期待できます。

Cloudflare Queues

D1同様にオープンベータ中のCloudflare Queues（以下、Queues）というキューイングのサービスも存在します。QueuesはWorkerに完全に統合されているので、Workerで重たい処理を行いたい場合に、その処理を非同期化するために簡単に使うことができます。例えば、リクエストに関するログ情報をR2バケットに書き込みたい場合、リクエストを受け付けたWorkerは、Queuesへログ情報を送信してすぐにレスポンスを返し、別のWorkerがQueuesからログ情報を受け取ってR2バケットに書き込む、といった形で処理を分解することが

^{注8} <https://blog.cloudflare.com/d1-open-beta-is-here/>

できます。このように、Queuesを使ったWorkerの実装は、Queuesに対してメッセージを送信するProducer Workerと、Queuesからメッセージを受信するConsumer Workerに分かれます。Consumer Worker側は特別なハンドラーの実装が必要ですが、それ以外の点については通常のWorkerと異なる点はほとんどなく、非常に簡単に実装できます。



本項で紹介したとおり、AWSやGCPなどの外部のインフラを使用せずに、Cloudflare単体でアプリケーションの構築を済ませることを可能にするプロダクトが登場してきています。今後、Cloudflareのプラットフォーム上でのアプリケーション開発がより行いやすくなっていくことは間違いないでしょう。

低コスト

Cloudflareのプランは無料プランと有料プランに分かれています。有料プランは月々5ドルで、製品ごとに無料枠も設定されています。有料プランの無料枠を超えた分は、従量課金となります。

無料プランの場合、Workerへのリクエストは1日10万件受け付け可能です。有料プランでは、1日なんと1,000万リクエストが無料枠に含まれており、それを超える分は100万リクエストごとに0.3ドルの課金となります^{注9}。これはあくまでWorkerへの1回の呼び出しごとのカウントで、実際にアプリケーションを構築した場合は1ユーザーあたりのWorkerの呼び出し数は複数回にわたることが予想されるので、1,000万ユーザーからのリクエストを5ドルの枠でさばけるということにはならないでしょう。しかしながら、十分に安価な価格設定と言えます。

Cloudflare Workersの主なユースケース

ここまで、Cloudflare Workersの特徴について見てきましたが、ここからは、実際の使い道について紹介します。

注9 2023年10月31日から有効となっています。
<https://blog.cloudflare.com/workers-pricing-scale-to-zero/>

リバースプロキシ

Cloudflare Workersの代表的な用途です。Workerを前段に立て、裏側のオリジンサーバに対してプロキシを行い、その中間でリクエスト・レスポンスを加工したり、キャッシュを行ったりします。yusukebeさんの「Cloudflare Workers プロキシパターン」というZennの記事^{注10}では、Basic認証プロキシ、リダイレクト、パスに基づくオリジンの振り分け、デバイス種類別のキャッシュ、HTMLタグの置換などのパターンが紹介されています。オリジンサーバの実装に手を加えることなく、前段で複雑な処理を行うことができるのは非常に魅力的です。リバースプロキシ用途では、いかにオリジンサーバへのリクエスト回数を減らして、負荷を軽減できるかという観点がポイントになるでしょう。Worker上で認証を行うパターンでは、認証に失敗した場合はオリジンサーバへリクエストが到達しない形の実装ができます。また、複数のパターンを組み合わせると、さらに強力な機能を実現できます。たとえば、認証プロキシとキャッシュを組み合わせると、認証のかかったページをエッジサーバでキャッシュすることも容易に実現可能です。リバースプロキシの実装にHono^{注11}などの軽量なWebフレームワークを組み合わせると、複雑なミドルウェアを自分で実装することなく利用可能な点も魅力的です。WebAssemblyを活用することで、Rustなどの言語によって記述したリバースプロキシを実装することも可能です。

Webアプリケーションサーバ

Workerをそのままアプリケーションサーバとして使用するパターンです。このパターンでは、裏側のオリジンサーバは存在せず、Worker上でリクエストを処理し、生成したレスポンスを返却します。先ほどのパターンでも紹介したHonoを使用すると、単純な文字列、JSON、HTMLなどを返却するWorkerを簡単に実装することができます。Honoのサードパーティのミドルウェア^{注12}を活用することで、OpenAPIやGraphQLのサーバを実装することもできます。

注10 <https://zenn.dev/yusukebe/articles/647aa9ba8c1550>

注11 <https://hono.dev/>

注12 <https://hono.dev/middleware/third-party>

Webアプリケーションサーバには、多くの場合、静的コンテンツを配信するためのストレージや、データベースなどが必要となりますが、この目的には前節の「Cloudflare プラットフォーム製品との柔軟な連携」で紹介した、Cloudflare プラットフォームの製品が活用できるでしょう。データベースに関しては、D1がまだベータ版ということもあり、実用的なプロダクトとしては、PlanetScaleのdatabase.js^{注13}やSupabaseのsupabase.js^{注14}など、サーバーレスのアプリケーションからデータベースへアクセスするためのライブラリが作成されていますので、これらを使用するのがよいでしょう。利用可能なデータベースの選択肢は、Cloudflare Workersのドキュメント「Connect to databases」^{注15}上で紹介されています。最近では、2023年5月にCloudflare WorkersがTCPソケットの利用に対応しました^{注16}ので、任意のデータベースがしくみ上利用可能となっています。ただし、Workerはリクエストのたびにアプリケーションを起動するしくみとなっているので、コネクションプールを持つことができない点が課題となっていました。この課題を解決するためのHyperdrive^{注17}というプロダクトが2023年9月に発表されています（5.1を参照）。新しい製品や機能の発表頻度から、Cloudflareが実用的なWebアプリケーションサーバをWorkerで実現可能にするために、非常に精力的に取り組んでいることがわかります。

フルスタックWebアプリケーション

Cloudflare Pages（後述）を使って、バックエンドからフロントエンドのアプリケーションまでのすべてをCloudflare上で構築するパターンです。Cloudflare Pagesは、Cloudflare上でフロントエンドのWebアプリケーションを作成するためのサービスで、その機能として提供されているPages FunctionsはWorkerをベースとしているため、ここまでに紹介したWorkerの機能を使用することができます。

注13 <https://github.com/planetscale/database-js>

注14 <https://github.com/supabase/supabase-js>

注15 <https://developers.cloudflare.com/workers/databases/connecting-to-databases/>

注16 <https://blog.cloudflare.com/workers-tcp-socket-api-connect-databases/>

注17 <https://blog.cloudflare.com/ja-jp/hyperdrive-making-regional-databases-feel-distributed-ja-jp/>

Cloudflare Pagesのアプリケーションは、静的コンテンツと、Pages Functionsを使用した動的コンテンツの組み合わせによって実現されます。たとえば、静的コンテンツとしてReactやVueで実装した簡素なアプリケーションを配置し、そこからPages Functions上で動作するHonoで実装されたAPIを呼び出すといった構成を行うことが可能です。

最近では、Pages上でSSR（サーバサイドレンダリング）の機能を備えたフルスタックなWebアプリケーションフレームワークを動かす例が増えています。とくに、ReactベースのフレームワークであるRemix^{注18}は採用例が増えてきています。Next.jsを動作させる例^{注19}も登場していますが、まだ安定していると

 言えない状況です。

1.1 まとめ

本稿では、Cloudflare Workersの特徴とユースケース、およびCloudflare プラットフォームの製品について紹介を行いました。1.2は、コードを交えつつ、Cloudflare WorkersおよびCloudflare Pagesの実際の使い方について紹介します。

注18 <https://remix.run/>

注19 <https://github.com/cloudflare/next-on-pages>

1.2 Cloudflare Workersをはじめよう

1.1 は、Cloudflare Workersの特徴と、どのような用途に使えるかについての概要を説明しました。1.2 では、実際にCloudflare Workersを動かしたうえで、Workerのプログラムの構成や、開発ツールであるWranglerについての説明を行います。

Cloudflare WorkersでHello World!

それでは、さっそくCloudflare Workersを動かしてみましょう。最初のWorkerとしては、リクエストを行うと“Hello World!”という単純なメッセージを返すものを作成します。

2023年5月に、create-cloudflare-cli（略してC3とも言います）というCLIツールが登場^{注1}し、Cloudflare Workersのプロジェクトの作成は非常に容易になりました。本記事は、C3によるGetting Startedのガイド^{注2}をベースとして行います。

事前準備

必要なものは次のとおりです。

- Cloudflare アカウント
- Node.js
- npm

執筆にあたって、Node.jsは最新のLTS（20.9.0）を、npmはバージョン10.2.3を使用しました。

注1 <https://blog.cloudflare.com/making-cloudflare-for-web/>

注2 <https://developers.cloudflare.com/workers/get-started/guide/>

Cloudflareアカウントの準備

Cloudflare アカウントは、以下のリンクから作成できます。

<https://dash.cloudflare.com/sign-up>

Cloudflare アカウントの作成は、個人であっても簡単に行えます（ここでは、アカウント登録フローの詳細については説明を行いません）。

Workerプロジェクトの作成

まず、Workerの開発に使うディレクトリを作成し、ターミナル上で、そのディレクトリ内に移動してください。ディレクトリに入ったら、`npm create cloudflare@latest` コマンドを入力してC3（create-cloudflare-cli）を起動してください。

次のように表示されたら、`y`を入力して先に進みます。

```
to install the following packages:
create-cloudflare@2.6.2
Ok to proceed? (y) y
```

2023年11月現在のC3（バージョン2.6.2）では、全部で3つのステップを実行してプロジェクトを作成します。

1つめのステップでは、まずディレクトリ名（兼アプリケーション名）を尋ねられます。今回は、`hello-world`と入力して先に進みましょう。

2つめのステップでは、どんな種類のアプリケーションを作成するか尋ねられます。今回は、「Hello World Worker」を選択しましょう。

3つめのステップでは、TypeScriptを使用するかどうかを尋ねられます。本稿では、説明を簡略化するためにTypeScriptなしでの説明を行いますので、Noを選択してください。

ここまで完了すると、プロジェクトの作成と、必要なライブラリのインストールが実行されます。

最後に、作成されたアプリケーションをデプロイするかについて尋ねられま

すが、ここではNoを選択しましょう。すると、現在の作業ディレクトリ内に、“hello-world”という名前のディレクトリが作成されていることが確認できます。

ローカルでのWorkerの起動

まずは、作成したWorkerのプロジェクトを、ローカルで実行してみましょう。

C3コマンドによって作成された“hello-world”ディレクトリに移動し、npm startコマンドを実行してください。すると、Cloudflare製の開発ツールであるWranglerが起動し、次のように表示されます。

```
[b] open a browser, [d] open Devtools, [l] turn off local mode, [c] clear console, [x] to exit
```

この状態で、bを入力しましょう。すると、ブラウザ上に“Hello World!”と表示されます。これで、ローカルでのWorkerの起動および動作確認は完了です。

ターミナルでcurlコマンドが使える環境の方は、`curl 127.0.0.1:8787`を実行することでも同様に“Hello World!”のメッセージが返ってくることが確認できます。Worker開発時の動作確認にブラウザを使うか、ターミナルを使うかはご自身で使いやすいと感じるほうをご利用ください。

Workerの編集

作成したWorkerの実装は、srcディレクトリ配下に格納されています。Workerプログラムの内容を見てみると、**リスト1**のように書かれているのが確認できます（コラム「インデントとタブ」参照）。

リスト1 Workerプログラムの内容

```
export default {
  async fetch(request, env, ctx) {
    return new Response('Hello World!');
  },
};
```

COLUMN

インデントとタブ

C3で作成したプロジェクトのPrettierフォーマッターの設定(.prettierrc)は、インデントにタブ文字を使用するuseTabsオプションが有効となっています。一般的に、JavaScriptおよびTypeScriptのプログラムでは半角スペース2つがインデントに使用される傾向にありますが、アクセシビリティの観点からタブ文字を使うべきではないかという議論^{注A}が行われています。インデントがタブ文字でも困らないと感じた方は、ぜひこの機会に試してみتهいかがでしょうか？

注A <https://sosukesuzuki.dev/posts/prettier-uses-tabs/>

npm startを実行したままで、このプログラムのHello World!の部分をHello Worker!に書き換えて保存し、先ほど開いたブラウザのページをリロードしてみましょう。すると、意図どおりにHello Worker!が返ってくることが確認できます。このように、Wranglerはプログラムの変更を検知して自動的にWorkerをロードし直すため、自分で停止・再開の操作を行わなくてもすぐに変更が反映されます。

Workerのデプロイ

続いて、Workerを実際にCloudflareにデプロイしてみましょう。npm run deployコマンドを実行してください。初めてCloudflareにデプロイする際は、wranglerコマンドから、Cloudflareアカウントの認証を求められます。

デプロイされるWorkerのURLは、hello-world.サブドメイン.workers.devとなります。サブドメインの部分は、デプロイの過程で入力を求められます。

デプロイが完了したら、ターミナルに表示されるURLを開いてみましょう。すると、先ほどローカルで試したとおりにHello Worker!の文字列が返ってくることが確認できます。

ここまでの、初めてのWorker作成からデプロイまでの一通りの作業は完了となります。

Workerプログラムの構成

続いて、先ほどリスト1に示したWorkerプログラムの構成を見てみましょう(再掲)。

```
export default {  
  async fetch(request, env, ctx) {  
    return new Response('Hello World!');  
  },  
};
```

Workerのプログラムは、ECMAScriptモジュールのデフォルトエクスポートによって公開される1つのオブジェクトによって構成されています。このオブジェクトには、fetchというキーにFetch Handlerと呼ばれる非同期関数が設定されていて、これがWorkerへのリクエストを処理する箇所となります。

Fetch Handlerは、Requestインスタンスを受け取って、Responseインスタンスを返却する非同期関数です。この、非常にシンプルな関数のインターフェースは、Web標準であるService Worker API^{注3}をベースとしています。実は、**リスト2**のように、Service Worker形式でWorkerのプログラムを書くこともできます。

リスト2 Service Worker形式のWorkerプログラム

```
addEventListener("fetch", (event) => {  
  event.respondWith(new Response("Hello World!"));  
})
```

しかしながら、Service Worker形式のWorkerには、Durable Objectsなどの機能を利用できないという制約があるため、基本的に使用せずに、C3コマンドが自動生成したとおりの形式を利用するのがよいでしょう。

さて、Fetch Handlerが受け取るenv引数は、Workerに対するバインディングを保持するオブジェクトです。ここには、Workerに対して設定された環境変数や、KV、R2などのサービスへアクセスするためのインスタンスがプロパティ

として設定されます。ctx引数は、Fetch Handlerがレスポンスを返したあともPromiseが解決するまで待つ目的に使うctx.waitUntil関数など、Handler上で特別な処理をしたいときに呼び出す関数が設定されているオブジェクトです。

Fetch HandlerはWorkerへのHTTPリクエストをトリガーとして起動するHandlerでしたが、実は、HTTPリクエスト以外の方法でWorkerを起動することもできます。それぞれの方法に対してのHandlerが用意されており^{注4}、現在利用可能なものとしては、Email Handler、Queue Handler、Scheduled Handlerなどがあります。これらもFetch Handlerと同様の形式で、default exportされるオブジェクトに、それぞれのキー（Email Handlerならばemail、Scheduled Handlerならばscheduled）を指定して記述します。

Email Handler

Email Handlerは、Email Workers^{注5}の機能を利用したいときに設定します。たとえば、特定のメールアドレス宛に送られてきたメールを、JavaScriptによって記述したルールで別々のメールアドレスへ転送することなどができます。受け取ったメールに対して、動的に組み立てたメッセージを自動で返信することもできます。単に、メールの受信をトリガーとしてJavaScriptのプログラムを実行できる機構と見ることもできるので、メールの受信時に外部のWeb APIへリクエストを送るといった処理も書くことができます。工夫次第でメールの活用の幅を大きく広げることができるでしょう。

Queue Handler

Queue Handlerは、Cloudflare Queuesから受け取ったメッセージを処理するために登録します。Cloudflare Queuesには、メッセージを送信するProducerと、メッセージを受信して処理するConsumerの概念が存在し、Queue Handlerによって登録されるのはConsumer側の処理です。Producer側の処理は、Fetch Handlerに限らず、各種Handlerから呼び出すことができます。

たとえば、Producer側のWorkerのwrangler.tomlで、あるQueueにMY_QUEUEというバインディングを設定したとき、このQueueへのメッセージ

注3 https://developer.mozilla.org/ja/docs/Web/API/Service_Worker_API

注4 <https://developers.cloudflare.com/workers/runtime-apis/handlers/>

注5 <https://developers.cloudflare.com/email-routing/email-workers/>

の送信はenv.MY_QUEUE.sendメソッドを呼び出すことで行えます。sendメソッドは、構造化複製アルゴリズムによってサポートされる任意のJavaScriptの値を送信することができます。ただし、送信可能な値の最大サイズは128KBとなっています。ここで送信されたメッセージを引数として、Consumer側のWorkerのQueue Handlerが呼び出されるといった形式となっています。

Scheduled Handler

Scheduled Handlerは、Cloudflare Workersを使って定期実行処理を実装したいときに登録します。Scheduled Handlerの実例の設定例については、次を参照してください。

```
# 毎時0分にCron Triggerを実行するwrangler.tomlの設定例
[triggers]
crons = ["0 * * * *"]
```

Scheduled Handlerによって登録された処理は、Cloudflare WorkersのCron Triggerによって呼び出されます。Cron Triggerの実行間隔は、crontabの形式でwrangler.tomlに指定します（Web上のUIからも設定が可能です）。

複数種類のHandlerをWorkerに対して同時に指定することもできます。これを利用して、定期実行したい処理の実装時のデバッグに、共通の処理を呼び出すFetch Handlerを使うこともできます（リスト3）。

リスト3 共通のrun関数を呼び出すScheduled HandlerとFetch Handlerの例

```
export default {
  async fetch(_, env) {
    await run(env);
    return Response.json({ "message": "success!" });
  },
  async scheduled(_, env, ctx) {
    ctx.waitUntil(run(env));
  },
};
```



このように、一見まったく異なる機能のように見える、Fetch/Queue/Email/

ScheduledのHandlerが1つのWorkerの基盤を共有することで、制約が非常に少ない、柔軟な開発体験が実現されています。



Wranglerとは

ここまで何度か言及してきましたが、Cloudflare Workersの開発では、基本的にCloudflare製のCLIツールであるWranglerが必要となります。

C3によって作成されるpackage.jsonには、初めからリスト4の内容が設定されていました。

リスト4 C3によって作成されるpackage.json（抜粋）

```
"scripts": {
  "deploy": "wrangler deploy",
  "dev": "wrangler dev",
  "start": "wrangler dev"
}
```

単純なWorkerを実装するだけであれば、このscriptsの設定を使ってnpm startと、npm run deployを実行するだけで開発からデプロイまでがほぼ完結します。しかしながら、Cloudflareの基盤をより活用していくには、Wranglerを直接使う必要があるでしょう。



Wranglerのインストール方法

Wranglerは、npm packageとして配布されています。基本的には、npm i -D wrangler コマンドで、プロジェクトローカルなdevDependenciesとしてインストールします。C3でプロジェクトを作成したときも、この方法によって自動的にインストールされます。プロジェクトローカルにインストールされたWranglerは、npx wrangler コマンドで呼び出します。グローバルにインストールしたい場合は、npm i -g wrangler コマンドを使いますが、同じプロジェクトを共有して開発するメンバーに、個別にWranglerコマンドをグローバルにインストールしてもらわずに済むので、基本的にはプロジェクトローカルなものを使うとよいでしょう。



Wranglerの設定ファイル「wrangler.toml」

まず、C3でプロジェクトを作成したときに自動で作成された、wrangler.tomlというファイルを見てみましょう（リスト5）。

リスト5 C3によって作成されるwrangler.tomlの内容

```
name = "hello-world"
main = "src/index.js"
compatibility_date = "2023-10-30"

# Variable bindings. These are arbitrary, plaintext strings (similar to
# environment variables)
```

初期状態では、Workerの基本設定と、Cloudflareの提供するほかのサービスへのバインディング方法のコメントが記載されています。

nameキーには、Workerの名前を設定します。C3を使った際に入力を求められた名前がここに設定されます。wrangler deployでワーカーをデプロイすると、ここで設定した名前から始まるドメインでワーカーへアクセス可能となります。

mainキーには、Workerのエントリーポイントとなるスクリプトのパスを指定します。ここで、mainのスクリプトからimportされている別のスクリプトのパスまで細かく指定する必要はありません。リスト6ではJavaScriptのファイルを指定していますが、mainキーにはTypeScriptのファイルを直接指定することもできます。

compatibility_dateは、後方互換性のない変更がCloudflare Workersのランタイムに入ったときに、古い挙動を維持し続けるために使われます。たとえば、2023年11月1日にプロジェクトを作成すると、compatibility_dateは2023-11-01に設定されます。同年11月15日に特定の機能に非互換な変更が入ったとしても、compatibility_dateを変更しなければその機能の挙動は変わりません。新しい挙動に切り替えたい場合は、compatibility_dateを最新の日時に手動で書き換えたうえで再度デプロイする必要があります。後方互換性のある変更は常にWorkerのランタイムに反映されているので、compatibility_dateを頻繁

に更新する必要はありません^{注6}。



Wranglerのサブコマンド

Wranglerのサブコマンドの中でとくに重要なのは、dev、deployの2つです。これらは、wrangler.tomlに設定された内容をもとに動作するので、細かいフラグを覚えたりする必要は基本的にありません。

そのほかによく使う便利なものとしては、tailサブコマンドがあります。wrangler.tomlの配置されているプロジェクト内でnpx wrangler tailを実行すると、デプロイされたWorkerのログをリアルタイムに閲覧することができます。

Worker以外のCloudflareのサービスを利用するためのコマンドも、Wranglerのサブコマンドとして提供されています。具体的には、kv、r2、queues、d1、pagesといったサブコマンドが存在します。たとえば、Cloudflare R2にバケットを作成して、ファイルをPutしたいというようなケースでは、npx wrangler r2 bucket create バケット名を実行し、npx wrangler r2 object put バケット名/キー file ローカルのファイルパスを実行します。このように、R2を使うが、Workerは使う必要がないようなケースでも、Wranglerを使うシーンはあるでしょう。



1.2 まとめ

C3を使ったCloudflare Workersプロジェクトの作成方法と、Workerプログラムの構成、各種Handlerの紹介、Wranglerコマンドおよびwrangler.tomlなどを紹介しました。この内容をもとにして、ぜひご自身でWorkerプロジェクトの作成・実行を試してみてください。

注6 compatibility_dateによって管理される後方互換性のない変更の一覧は、CloudflareのCompatibility datesのドキュメントのページ下部に記載されています。
<https://developers.cloudflare.com/workers/configuration/compatibility-dates/>

1.3 作って学ぶ Cloudflare Pages

1.2では、Cloudflare Workersのプロジェクトを作成して、実際にHello World!のメッセージを返すWorkerを動かしてみました。Cloudflare Workersでは、任意のJavaScriptのコードを実行できるので、HTMLを動的に組み立てて返すような単純なWebアプリケーションであれば十分作成可能でしょう。

しかしながら、近年のWebアプリケーションでは、単にサーバサイドからHTMLを返すだけにとどまらないケースが増えています。とくに、ReactやVueなどをベースにしたWebアプリケーションフレームワークを利用して、リッチな体験をクライアントサイドで提供する形式に遭遇する機会は多いでしょう。こうしたケースでは、サーバサイドのアプリケーションでデータベースやストレージに対する書き込み・読み取りの処理を行い、クライアントサイドのアプリケーションでサーバサイドのアプリケーションから受け取った結果を画面に表示するといった形で分担を行います。

Cloudflare Workers単体では、基本的にサーバサイドのアプリケーションしか作成できませんが、Cloudflare Pagesを使うことでクライアントサイドのアプリケーションも含んだ、フルスタックなWebアプリケーションをCloudflareスタック上で構築することが可能となります。

ここでは、このCloudflare Pagesの特徴および、実際の使い方について紹介します。

Cloudflare Pagesとは

Cloudflare Pagesは、前述のとおり、フルスタックなWebアプリケーションをCloudflare上で構築するためのサービスです。クライアントサイドのアプリケーション用には、HTML、CSS、(ブラウザ上で動作させるための)JavaScriptなど、静的なコンテンツをアップロードして配信します。サーバサイドのアプリケーション用には、Pages FunctionsというCloudflare Workersをベースとしたサーバーレス関数を使います。Pages Functionsは、Cloudflare Workers

と同様にD1、R2などのCloudflareスタックのサービスにアクセスすることが簡単にできます。

Cloudflare Pagesは、基本的には静的なコンテンツを配信するものの、一部の処理を動的に行いたいようなケースに非常によく合うでしょう。たとえば、Webサービスのランディングページを作成する際に、ページのコンテンツは静的に作成し、フォームの送信部分にPages Functionsを使うといった構成が考えられます。Pages Functionsはサーバーレス関数ですので、専用のインスタンスを常時稼働し続ける必要がなく、コストが非常に安価となります。また、Workerと共通の基盤を使用しているため、起動速度も高速です。

最近では、以前までNode.jsでの動作を前提としていたWebアプリケーションフレームワークが、Cloudflare Workersを含む非Node.js環境での動作をサポートするケースが増えています。こうしたフレームワークを利用することで、ほとんど静的コンテンツで構成されているような単純なWebサイトだけでなく、動的コンテンツをサーバサイドでレンダリングするなどの機能を持つ、フルスタックなWebアプリケーションをCloudflare Pages上でホストすることも可能となっています。

Cloudflare Pagesの特徴

Cloudflare Pagesは、次のような特徴を備えています。

- Git連携
- スケーラブル・高パフォーマンス
- 各種Webアプリケーションフレームワークのサポート

Git連携

Cloudflare Pagesは、GitHubおよびGitLabと連携することができます。連携を行うことで、Gitリポジトリへのプッシュをトリガーとした自動デプロイが可能となります。

自動デプロイの設定では、どのブランチをデプロイに使うか(例: main ブランチ)や、ビルドに使うコマンドなどを自由に設定することができます。

さらに、プレビューブランチ（デフォルトではmain以外のすべてのブランチが対象）を設定すると、GitHubのリポジトリに対してプルリクエストを作成するたびに、自動でプレビューデプロイが行われるようになります^{注1}。プレビューデプロイされたページは、対象のプルリクエストに対してコミットが行われるたびに内容が更新されるため、レビューによる動作確認が容易に行えます。また、プレビューデプロイのURLは、プルリクエストが作成されるたびに新たに発行されるため、複数の開発が同時に進んでいたとしても、問題なく運用することができます。Cloudflare Access^{注2}と組み合わせることで、プレビューデプロイの結果を特定のユーザーだけから閲覧可能にするよう制限することも可能です。

スケーラブル・高パフォーマンス

Cloudflare Workers同様、Cloudflare Pagesはスケーラブルなくみとなっています。静的コンテンツの配信はCloudflareの得意とする領域ですし、動的コンテンツの処理もエッジサーバ上でスケーラブルな形で行います。Pages Functionsと組み合わせて使うCloudflareのサービス群もスケールしやすいしくみとなっています。特別な考慮を過度に行うことなく、世界中のあらゆる地点から高速にアクセス可能なWebサイトを構築できるのは非常に魅力的です。

各種Webアプリケーションフレームワークのサポート

Cloudflare Pagesは、あらゆるWebアプリケーションフレームワークおよび静的サイトジェネレーターをサポートしており、それぞれの導入方法についてのガイド^{注3}が公式サイトに記載されています。Webアプリケーションフレームワークとしては、Remix、Astro、Gatsby、Next.js、Nuxt.js、Svelteなどが利用可能です。静的サイトジェネレーターとしては、Hugo、Jekyll、Docusaurusなどが利用可能です。

Cloudflare PagesはWebアプリケーションのビルドを行う機能を持ってお

^{注1} GitLabについてはドキュメントに記載されていることを確認できませんでした。

^{注2} <https://developers.cloudflare.com/cloudflare-one/policies/access/>

^{注3} <https://developers.cloudflare.com/pages/framework-guides/>

り、Git連携で変更がプッシュされたときなどに、事前に指定したビルドコマンドが自動的に実行されます。ビルドには、事前にインストールされているいくつかのプログラミング言語のランタイムや周辺ツールが利用可能となっています。プログラミング言語ランタイムとしては、Go、Node.js、Python、Rubyが利用可能です。周辺のツールとしては、npm、gem、pipなどが利用可能です。事前にインストールされているすべてのツールに関しては、ドキュメントをご覧ください^{注4}。

Cloudflare PagesのHello World!

それでは、実際にCloudflare Pages上にWebサイトを構築してみましょう。まずは、最もシンプルな構成で構築を進めます。通常、Cloudflare Pagesのプロジェクト作成は、C3（create-cloudflare-cli）コマンドを使用して各種Webアプリケーションフレームワークのテンプレートから行いますが、今回は、Cloudflare Pagesのプロジェクト構成の理解のために、あえて素の状態からプロジェクトを構築してみます。

事前準備

1.2同様に、次のものが必要となります。

- Cloudflare アカウント
- Node.js
- npm

プロジェクトの作成

まず、ターミナル上でPagesの開発に使うディレクトリに移動してください。移動したら、次のような構成でディレクトリ、ファイルを作成します。作成はどのような方法で行ってもかまいません。初めは、index.htmlの中身は完全に空の状態にしてください。

^{注4} <https://developers.cloudflare.com/pages/platform/language-support-and-tools/#supported-languages-and-tools>

```
hello-world-pages
├── public
│   └── index.html
```

上記のディレクトリおよびファイルは、次のような役割となります。

ディレクトリ・ファイル	役割
hello-world-pagesディレクトリ	プロジェクトのルートディレクトリ
publicディレクトリ	ブラウザ上で動作するHTMLやCSS、JavaScriptなどの静的なコンテンツを設置するディレクトリ。Cloudflare Pagesは特定のディレクトリ名を使うことを強制しないので、今回はわかりやすい名前としてpublicを使用しています。
index.html	Cloudflare Pagesで作成したWebサイトのトップページ

ディレクトリ、ファイルの作成が完了したら、index.htmlの内容を**リスト1**のように変更してください。

リスト1 index.htmlの変更内容

```
<!doctype html>
<html>
<head><title>Hello World example</title></head>
<body>
Hello World!
</body>
</html>
```

index.htmlの編集が完了したら、さっそくCloudflare Pagesにデプロイしていきます。

Cloudflare Pagesへのデプロイ

デプロイのために、Wranglerをプロジェクトローカルにインストールしましょう。

まずは、`npm init -y`でpackage.jsonを作成し、その内容を**リスト2**のように書き換えてください。

リスト2 package.jsonに記載するWranglerのローカルインストール用設定

```
{
  "name": "hello-world-pages",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "deploy": "wrangler pages deploy ./public"
  }
}
```

編集が完了したら、`npm i -D wrangler`でdevDependenciesとしてWranglerをインストールします。これで、Cloudflare Pagesへデプロイするための準備は完了です。

それでは、実際にデプロイを行ってみましょう。package.jsonに登録したdeployスクリプトを呼び出すために、`npm run deploy`を実行します。初めてWranglerを使う方は、ここでまず自身のCloudflareアカウントに対する認証を行ってください。

`npm run deploy`を実行すると、次のように、デプロイ先のプロジェクトを新しく作るか、すでに存在しているものを使うか尋ねられます。今回は、新しいプロジェクトを作成するので、「Create a new project」を選択し、その後、プロジェクト名としてhello-world-pagesを指定してください。

```
No project selected. Would you like to create one or use an existing ☒ project?
> Create a new project
  Use an existing project
✓ Enter the name of your new project: ... hello-world-pages
```

続いて、本番環境のブランチ名を尋ねられるので、デフォルト値のmainのまま進んでください。

「Deployment complete!」というメッセージが表示されたら、Cloudflare Pagesへのデプロイは完了です。完了後に、デプロイされたページにアクセスするためのURLがターミナル上に表示されます。このURLにアクセスすると、期待どおりに「Hello World!」というメッセージが画面上に表示されることが確認できます。デプロイ直後はURLを開いてもアクセスに失敗し閲覧できないこ

とありますが、その場合は5〜10分程度、時間を置いてから再度アクセスしてください。

初めてのPages

静的なWebサイトを作成することに成功したので、次は動的なコンテンツを追加してみましょう。具体的には、アクセスカウンターを追加してみます。Cloudflare Workers KV（以下、KV）を使用して、先ほどのページに一度アクセスするごとに、1つカウントを行うようにします。ここで注意が必要ですが、KVは結果整合であり、強整合ではないので、アクセスカウンターの増加数が意図しない結果となる場合があります。強整合が必要なケースでは、KVではなくDurable Objectsを使用してください。

KVの設定の追加

まずは、プロジェクトのルートディレクトリにwrangler.tomlを追加します。wrangler.tomlは、1.2で紹介した、Cloudflare Workersのプロジェクトの設定や、各サービスとのバインディングを行うためのファイルです。wrangler.tomlは、Pages Functionsの設定にも使われます。今回は、KVの機能をPages Functionsのコードから呼び出すために作成します。

wrangler.tomlは、次の内容で作成してください。

```
name = "hello-world-pages"
compatibility_date = "2023-12-10"
```

続いて、アクセスカウンターの情報を登録するためのKV namespaceを作成します。

```
$ npx wrangler kv:namespace create hello_world_pages_counter
```

KV namespaceの作成が完了すると、次のようなメッセージが表示されるので、**id**キーで示されている**KV namespace ID**の内容を控えておいてください。

```
{ binding = "hello_world_pages_counter", id = "KV namespace ID" }
```

次に、wrangler.tomlの末尾に**リスト3**の内容を追記し、**id**キーの指定内容を先ほど控えたものに差し替えてください。ここでbindingキーに指定されたKVがWorkerやPages Functionsのコード内からKV namespaceにアクセスする際に使う、envオブジェクト上のプロパティ名となります。

リスト3 KVの設定

```
kv_namespaces = [
  { binding = "KV", id = "KV namespace ID" }
]
```

ここまでの作業で、KVの設定は完了となります。

Pages Functionsの実装

続いて、Pages Functionsの実装を行います。Pages Functionsのファイルは、プロジェクトのルートディレクトリ直下にfunctionsというディレクトリを作成したうえで、その中に設置する必要があります。functionsディレクトリを作成したら、次のような構成でcounter.jsというファイルを設置しましょう。

```
hello-world-pages
├── functions
│   └── counter.js
```

Pages Functionsは、Cloudflare Workersと共通の基盤を使用していますが、ハンドラーの設定方法は大きく異なります。Cloudflare Workersは、各種ハンドラーを設定したオブジェクトをdefault exportで公開する形式でしたが、Pages Functionsでは、onRequestという名前の関数をexportする形式でハンドラーを設定します。

今回のアクセスカウンターの例では、counter.jsを**リスト4**のように実装します。

リスト4 アクセスカウンターの作成における counter.js の実装

```
export async function onRequest(context) {
  // KVの"count"キーの値を取得する
  const value = await context.env.KV.get("count");
  // KVから取得した値を数値に変換する。数値に変換できなければ、値を0とする
  const currentCount = Number.isNaN(Number(value)) ? 0 : Number(value);
  // カウントを1加算する
  const count = currentCount + 1;
  // 加算結果をKVに保存する
  await context.env.KV.put("count", count);
  // 加算結果を"count"キーに設定したJSONレスポンスを返す
  return Response.json({ count: count });
}
```

 Pages Functionsのルーティング

ファイル名の counter.js は、そのままルーティングに使用されます。たとえば Cloudflare Pages によって作成された Web サイトの URL を example.com としたとき、/functions/counter.js の呼び出しは example.com/counter へリクエストすることで行えます。公式ドキュメントのルーティングのガイド^{注5}では、次のように示されています。

File path	Route
/functions/index.js	example.com
/functions/helloworld.js	example.com/helloworld
/functions/howdyworld.js	example.com/howdyworld
/functions/fruits/index.js	example.com/fruits
/functions/fruits/apple.js	example.com/fruits/apple
/functions/fruits/banana.js	example.com/fruits/banana

[x].js のように角括弧を使ったファイル名を利用することで、複数の URL に対してマッチさせることもできます。たとえば、/functions/users/[userId].js という URL を使用した場合、/users/1 という URL でアクセスしても呼び出しが行われます。userId の値は、onRequest ハンドラーの context 引数を經由して取得することができます。ルーティングの詳細については、公式のルーティン

注5 <https://developers.cloudflare.com/pages/platform/functions/routing/>

グのガイドをご参照ください（上記注30）。

 Pages Functionsの呼び出し

最後に、/public/index.html を編集して、アクセスカウンターの結果を表示できるようにしましょう。body タグの内容を、**リスト5**のように書き換えてください。

リスト5 アクセスカウンターの結果を表示する設定

```
<div>Hello World!</div>
<div id="counter"></div>
<script type="module">
  const res = await fetch("/counter");
  const result = await res.json();
  const counter = document.getElementById("counter");
  counter.textContent = `Count: ${result.count}`;
</script>
```

このコードでは、先ほど作成した /functions/counter.js の Pages Function にリクエストを送り、レスポンスから読み取ったオブジェクトの "count" キーの内容を "counter" の ID 付きの div 要素の内容に設定しています。

こちらのコードの動作確認を行うために、package.json の "scripts" に次のスクリプトを追加しましょう。wrangler pages dev は、Cloudflare Pages をローカル開発環境で動かすためのコマンドです。

```
"dev": "wrangler pages dev --compatibility-date=2023-12-10 ./public"
```

追加が完了したら、`npm run dev` でスクリプトを起動しましょう。起動が完了したら、**b** キーを押すか、localhost:8788 にアクセスすることで、KV によるアクセスカウンターがブラウザ上で動作していることを確認できます。



1.3 まとめ

1.3 は、Cloudflare Pages の特徴および使い方について紹介しました。紹介した例は、最小限の構成でCloudflare Pagesを使いたい場合や、別プラットフォームからの移管時に活用できると思います。基本的には、C3 コマンドを使ってRemixなどのテンプレートから作成するのがおすすめです。

Cloudflare Pages は、Cloudflare Workersの良さをそのまま活かしつつ、作成できるアプリケーションの幅を大きく広げてくれる、非常に魅力的なサービスです。ぜひ、この内容を参考に、Cloudflare Pagesのアプリケーション作成を試してみてください。